AI-Powered Schema Inference for Automatic Data Ingestion

Tom Tang
Edge Vector Foundation

2026

Abstract

A system and method for automatically ingesting arbitrary file inputs into a schema-enforcing database without manual schema design or data mapping. The system implements a multi-tier AI pipeline that receives an arbitrary file (including but not limited to documents, images, audio, code, PDFs, and spreadsheets), converts the file to structured JSON through a four-tier conversion pipeline with automatic fallback, infers or matches a schema from a global registry using embedding-based similarity detection, and generates typed database mutations that conform to the matched or newly created schema's field definitions, types, and constraints. The four-tier conversion pipeline comprises: (1) local deterministic parsing for structured formats such as JSON, CSV, YAML, and TOML, requiring zero large language model (LLM) cost; (2) vision model processing for image formats, wherein an LLM extracts structured data from visual content; (3) text extraction followed by LLM structuring for PDF and unknown text formats; and (4) chunked parallel LLM processing with merge for files exceeding a size threshold, wherein large files are split at format-aware boundaries (function and class boundaries for code, paragraph and section boundaries for prose, record boundaries for data), processed in parallel, and merged with deduplication and conflict resolution. The system compares LLM-generated schemas against a global schema registry using embedding similarity to identify existing matching schemas, preventing schema proliferation. When multiple similar schemas exist, the system ranks candidates and recommends the best match. Once a schema is identified or created, the system generates typed field mutations that conform to the schema's field definitions, types, and constraints. The entire pipeline is idempotent: re-ingesting the same file produces the same schema and mutations.

1 Introduction

This work relates generally to automated data ingestion systems, and more particularly to methods and systems for using artificial intelligence to infer database schemas from arbitrary file inputs, match inferred schemas against a global registry of existing schemas using embedding similarity, and automatically generate typed database mutations from unstructured or semi-structured file content, enabling any file to be ingested into a schema-enforcing database without manual schema design, data mapping, or format-specific ingestion code.

2 Background

2.1 The Data Ingestion Problem

Modern databases enforce schemas: every record must conform to a predefined structure of field names, types, and constraints. Ingesting data from arbitrary file formats into such databases requires three manual steps that create friction and cost:

Format-specific parsing. Each file format (PDF, DOCX, XLSX, PNG, MP3, source code) requires a dedicated parser. Organizations maintain dozens of parsers, each requiring ongoing maintenance as formats evolve. When a new format is encountered, ingestion stalls until a parser is written.

Schema design. A human must examine the content of a file, decide which fields to extract, choose types for each field, and design a schema that captures the relevant information. This requires domain expertise and is inherently subjective: two humans examining the same file may produce different schemas.

Data mapping. Once a schema exists and the file is parsed, a human or hand-written program must map the parsed data to the schema's fields. This mapping is fragile: changes to the file format, the schema, or the mapping logic can break ingestion silently.

These three steps are repeated for every new file type, every new schema, and every new data source. The result is a proliferation of bespoke ingestion pipelines, each tightly coupled to a specific format-schema pair.

2.2 The Schema Proliferation Problem

In systems with many users or data sources, the same conceptual entity (e.g., a contact, an invoice, a medical record) is frequently represented by dozens of nearly identical but subtly different schemas. Each user or team creates their own schema without awareness of existing schemas that capture the same information. This proliferation creates data silos: records conforming to different schemas cannot be queried together, compared, or aggregated without manual reconciliation.

Existing approaches to schema management rely on human curation (a database administrator maintains a canonical schema catalog) or rigid naming conventions (all schemas must follow a prescribed format). Neither approach scales to the diversity and volume of data produced by modern systems.

2.3 Limitations of Prior Art

ETL (Extract-Transform-Load) pipelines are the industry standard for data ingestion. Each ETL pipeline is manually authored for a specific source-format-to-target-schema pair. ETL pipelines do not generalize: a pipeline built for CSV-to-contact-schema cannot ingest a PDF business card or a PNG screenshot of a contact list. ETL pipelines require ongoing maintenance and provide no mechanism for schema discovery or similarity detection.

Schema-on-read systems (e.g., data lakes) defer schema enforcement to query time. While this eliminates the upfront schema design step, it merely postpones the problem: queries still require knowledge of the data's structure, and inconsistencies between files are discovered at query time rather than at ingestion time, often after downstream systems have consumed malformed data.

Optical character recognition (OCR) and document parsing systems extract text from images and documents but do not infer schemas. The output is unstructured text that still requires manual mapping to database fields.

LLM-based data extraction systems use large language models to extract structured data from text. However, existing systems treat each extraction as independent: they do not maintain a global schema registry, do not detect similarity between inferred schemas, and do not prevent schema proliferation. They also lack format-aware chunking strategies for large files, instead relying on naive token-count splitting that breaks semantic units.

None of these approaches provide a unified pipeline that handles arbitrary file formats, infers schemas using AI, matches inferred schemas against a global registry to prevent proliferation,

generates typed mutations conforming to the matched schema, and does so idempotently with format-aware processing strategies that minimize LLM cost for files that can be parsed deterministically.

3 SUMMARY

The present invention provides a system and method for automatically ingesting arbitrary file inputs into a schema-enforcing database. The invention comprises:

1. **Four-tier file conversion pipeline.** A multi-tier pipeline converts arbitrary files to structured JSON, selecting the most efficient conversion strategy based on the file's format:
   (a) **Tier 1 — Local deterministic parsing**: Files in structured formats (JSON, CSV, YAML, TOML) are parsed deterministically using local parsers with zero LLM cost. The output is structured JSON that preserves the original data's types and hierarchy.
   (b) **Tier 2 — Vision model processing**: Image files (JPEG, PNG, GIF, WebP) are submitted to a vision-capable LLM that extracts structured data from the visual content, producing a JSON representation of the information depicted in the image.
   (c) **Tier 3 — Text extraction followed by LLM structuring**: Files that contain extractable text but are not in a structured format (PDF via a rendering library, unknown text formats) have their text extracted first, then submitted to an LLM that structures the extracted text into JSON.
   (d) **Tier 4 — Chunked parallel LLM with merge**: Files exceeding a configurable size threshold (default 128KB) are split into semantically meaningful chunks using format-aware splitting strategies, processed in parallel by independent LLM invocations, and the resulting JSON outputs are merged with deduplication and conflict resolution.

2. **Schema inference and similarity matching.** The structured JSON output from the conversion pipeline is analyzed to infer a schema (field names, types, and constraints). The inferred schema is compared against a global schema registry using embedding-based similarity detection. If a sufficiently similar schema exists, the system recommends adopting the existing schema rather than creating a new one. When multiple similar schemas exist, the system ranks candidates by similarity score and recommends the best match, preventing schema proliferation while preserving schema diversity for genuinely distinct data types.

3. **Automatic mutation generation.** Once a schema is identified (either matched from the registry or newly created), the system generates a set of typed field mutations that conform to the schema's field definitions, types, and constraints. Each mutation specifies the target field, the value to write, and the value's type, ready for direct submission to the schema-enforcing database without manual mapping.

4. **Format-aware chunking.** Large files are split at semantically meaningful boundaries rather than arbitrary byte offsets: code files are split at function and class boundaries using syntax-aware parsing (tree-sitter), prose documents are split at paragraph and section boundaries, and structured data files are split at record boundaries. This ensures that each chunk is a self-contained semantic unit that an LLM can process independently without losing context.

5. **Idempotent pipeline.** The entire pipeline is deterministic and idempotent: re-ingesting the same file produces the same structured JSON, the same schema match or creation, and

the same set of mutations. This property enables safe retries, deduplication of ingested data, and verification of pipeline correctness.

4 DETAILED DESCRIPTION

4.1 1. System Architecture Overview

The system comprises six components (see FIG. 1):

(a) A file intake service that receives arbitrary file inputs, determines their format through content inspection (MIME type detection and magic byte analysis), and routes each file to the appropriate conversion tier.

(b) A four-tier conversion engine that transforms files into structured JSON using the most efficient strategy available for each format, falling back to more expensive tiers only when cheaper tiers cannot handle the format.

(c) A schema inference module that analyzes structured JSON to produce a candidate schema comprising field names, inferred types, and structural constraints.

(d) A global schema registry that stores all known schemas and their embedding representations, enabling similarity-based lookup and preventing schema proliferation.

(e) A mutation generator that takes a schema and structured JSON data and produces a set of typed field mutations conforming to the schema's definitions.

(f) A schema-enforcing database that validates and stores mutations, rejecting any mutation that does not conform to the target schema's field definitions, types, and constraints.

4.2 2. Four-Tier Conversion Pipeline

The conversion pipeline implements a tiered strategy that minimizes cost and latency by using the cheapest conversion method that can handle a given file format (see FIG. 2). The tiers are evaluated in order; processing falls through to the next tier only when the current tier cannot handle the input.

4.2.1 2.1 Tier 1: Local Deterministic Parsing

Files in self-describing structured formats are parsed locally without invoking any LLM. Supported formats include:

- **JSON**: Parsed directly. The file is already structured JSON and requires no conversion.
- **CSV**: Parsed into an array of objects, with column headers as field names and values as strings (type inference applied post-parse).
- **YAML**: Parsed into equivalent JSON representation, preserving the document's hierarchical structure.
- **TOML**: Parsed into equivalent JSON representation, preserving typed values (integers, floats, booleans, dates).

Tier 1 is deterministic, has zero LLM cost, and produces identical output for identical input. The parser selection is based on file extension and content-type header, with fallback to content inspection (attempting each parser and accepting the first that succeeds without error).

4.2.2 2.2 Tier 2: Vision Model Processing

Image files are submitted to a vision-capable large language model. The system constructs a prompt that instructs the model to examine the image and extract all structured information visible in the image, producing a JSON object with descriptive field names and typed values.

Supported formats include JPEG, PNG, GIF, and WebP. The vision model receives the raw image bytes (base64-encoded) along with a system prompt that specifies the desired output format (JSON), the requirement for descriptive field names, and guidelines for type inference (numbers as numbers, dates as ISO 8601 strings, booleans as booleans).

The vision model's JSON output is validated for syntactic correctness. If the output is not valid JSON, the system retries with an error-correction prompt up to a configurable maximum number of retries.

4.2.3 2.3 Tier 3: Text Extraction + LLM Structuring

Files that contain extractable text but are not in a structured format are processed in two stages:

Stage 1: Text extraction. The system extracts raw text from the file. For PDF files, a rendering library (pdfium) extracts text content while preserving reading order. For unknown text formats, the system reads the file as UTF-8 text, falling back to common encodings (Latin-1, UTF-16) if UTF-8 decoding fails.

Stage 2: LLM structuring. The extracted text is submitted to an LLM with a prompt instructing the model to identify the structured information in the text and produce a JSON representation. The prompt includes guidelines for field naming, type inference, and handling of ambiguous or missing values.

If the extracted text exceeds the LLM's context window, processing falls through to Tier 4 (chunked parallel processing).

4.2.4 2.4 Tier 4: Chunked Parallel LLM with Merge

Files exceeding a configurable size threshold (default 128KB) or whose extracted text exceeds the LLM's context window are processed using chunked parallel processing:

Step 1: Format-aware chunking. The file is split into semantically meaningful chunks using format-specific splitting strategies:

- **Source code**: Split at function and class boundaries using syntax-aware parsing via tree-sitter. Each chunk contains one or more complete syntactic units (functions, classes, modules). Import statements and module-level declarations are duplicated into each chunk to preserve context.
- **Prose documents**: Split at paragraph or section boundaries. Section headers are preserved with their content. Cross-references within a chunk are preserved; cross-references to other chunks are noted as metadata.
- **Structured data**: Split at record boundaries. For CSV files, the header row is duplicated into each chunk. For JSON arrays, each chunk contains a contiguous subset of array elements. For nested structures, splitting occurs at the outermost array level.

Step 2: Parallel LLM processing. Each chunk is submitted to an independent LLM invocation with the same structuring prompt. All chunk invocations execute in parallel, reducing wall-clock time proportionally to the number of chunks (bounded by available concurrency).

Step 3: JSON merge with deduplication and conflict resolution. The JSON outputs from all chunk invocations are merged into a single JSON document using the following strategy:

- **Array fields**: Concatenated across chunks, then deduplicated by value equality. Order is preserved based on chunk order (which corresponds to the original file order).
- **Object fields**: Merged by key. When the same key appears in multiple chunks with different values, the system applies a conflict resolution strategy: for scalar values, the value from the earliest chunk (by file position) takes precedence; for nested objects, recursive merge is applied; for arrays, concatenation and deduplication as above.
- **Metadata fields**: Fields describing the file as a whole (title, author, date) are extracted from the first chunk and validated against subsequent chunks. Conflicts are resolved by majority vote across chunks.

4.3 3. Schema Inference

The schema inference module analyzes the structured JSON output from the conversion pipeline and produces a candidate schema. The inference process examines:

- **Field names**: Extracted directly from the JSON object's keys. Nested objects produce dot-separated compound field names (e.g., `address.city`).
- **Field types**: Inferred from the JSON values. Numeric strings are tested for numeric type promotion. Date-formatted strings are identified using a library of common date patterns. Boolean-like strings ("true", "yes", "1") are identified and typed as boolean.
- **Array fields**: The element type is inferred from the union of all element types. Homogeneous arrays are typed as `Array<T>`. Heterogeneous arrays are typed as `Array<Union<T1, T2, ...>>`.
- **Optional fields**: Fields that appear in some records but not others (for array-of-objects inputs) are marked as optional.
- **Constraints**: Value ranges, string lengths, and enumeration sets are inferred from the observed data and recorded as schema constraints.

The candidate schema is represented as a structured object containing field definitions (name, type, optionality, constraints) and metadata (inferred description, source file format, creation timestamp).

4.4 4. Schema Similarity Detection and Matching

The candidate schema is compared against all schemas in the global schema registry to determine whether an existing schema captures the same or substantially similar information (see FIG. 2).

4.4.1 4.1 Embedding Generation

The candidate schema is serialized to a canonical string representation (sorted field names, normalized types) and submitted to an embedding model that produces a dense vector representation. The global schema registry stores precomputed embedding vectors for all registered schemas.

4.4.2 4.2 Similarity Search

The candidate schema's embedding vector is compared against all stored embeddings using cosine similarity. Schemas exceeding a configurable similarity threshold (default 0.85) are returned as candidates, ranked by similarity score.

4.4.3 4.3 Schema Recommendation

When one or more candidate schemas exceed the similarity threshold, the system recommends the highest-scoring match. The recommendation includes:

- The matched schema's identifier, name, and description.
- The similarity score.
- A field-by-field comparison showing which fields in the candidate schema map to which fields in the matched schema, which fields are present in one but not the other, and which fields have compatible but non-identical types.
- A recommendation action: "adopt" (similarity above 0.95, near-identical schemas), "review" (similarity between 0.85 and 0.95, similar but not identical), or "create new" (no schema above the threshold).

When the recommendation is "adopt," the system proceeds with the matched schema. When the recommendation is "review," the system presents the comparison to the user for a decision. When the recommendation is "create new," the system registers the candidate schema in the global registry with its embedding vector.

4.4.4 4.4 Schema Registration

New schemas are registered in the global registry with:

- A unique schema identifier.
- The schema definition (field names, types, constraints).
- The embedding vector for future similarity searches.
- Metadata (creation timestamp, source file format, creator identity).

Registration is atomic: the schema is either fully registered (definition and embedding) or not registered at all. Concurrent registrations of similar schemas are serialized through optimistic concurrency control with conflict detection.

4.5 5. Automatic Mutation Generation

Once a schema is identified (matched or newly created), the mutation generator produces a set of typed field mutations from the structured JSON data.

For each field defined in the schema, the generator:

1. Locates the corresponding value in the structured JSON, using field name matching and the field mapping produced during schema matching (for adopted schemas where field names may differ slightly).

2. Validates the value against the field's type definition. Type coercion is applied where safe (e.g., string "42" to integer 42 when the schema specifies an integer field). Values that cannot be coerced to the target type are rejected with an error.
3. Constructs a typed mutation specifying the target schema, the target field name, the coerced value, and the value's type.

The output is an ordered list of mutations that, when applied to the schema-enforcing database, creates a new record conforming to the schema. Optional fields with no corresponding value in the JSON are omitted from the mutation list. Required fields with no corresponding value cause the entire mutation set to be rejected (no partial mutations are generated).

4.6 6. Idempotency

The pipeline guarantees idempotency through the following mechanisms:

- **Deterministic Tier 1 parsing**: Local parsers produce identical output for identical input.
- **Deterministic LLM prompts**: All LLM invocations use temperature 0 and a fixed system prompt. While LLM outputs are not perfectly deterministic, the combination of temperature 0 and structured output constraints (JSON schema enforcement) produces identical output for identical input in practice.
- **Content-addressed file identification**: Each file is identified by a cryptographic hash (SHA-256) of its contents. The system maintains a mapping from content hash to previous pipeline output. If the same file is submitted again, the cached output is returned without re-executing the pipeline.
- **Deterministic schema matching**: The embedding model produces identical vectors for identical inputs, and the similarity search is deterministic (cosine similarity with fixed threshold and deterministic tie-breaking by schema registration timestamp).
- **Deterministic mutation generation**: The mutation generator is a pure function of the schema and the structured JSON data. Given the same schema and the same JSON, it produces the same mutations.

4.7 7. Error Handling and Fallback

The pipeline implements structured error handling at each stage:

- **Tier fallback**: If a tier fails (parser error in Tier 1, invalid JSON output from LLM in Tiers 2–4), the system falls through to the next tier. Tier 1 failure (not a supported structured format) is not an error; it is the expected path for most file types. Tier 2–4 failures (LLM produces invalid output) trigger retries with error-correction prompts before falling through.
- **Schema matching failure**: If the embedding service is unavailable, the system creates a new schema without similarity matching. The schema is flagged for retroactive matching when the service recovers.
- **Mutation generation failure**: If a required field cannot be populated from the structured JSON, the entire mutation set is rejected. No partial mutations are submitted to the database.

All errors are logged with the file's content hash, the tier that failed, the error details, and a timestamp. The error log enables retroactive diagnosis and re-processing of failed files.

5 FIGURES

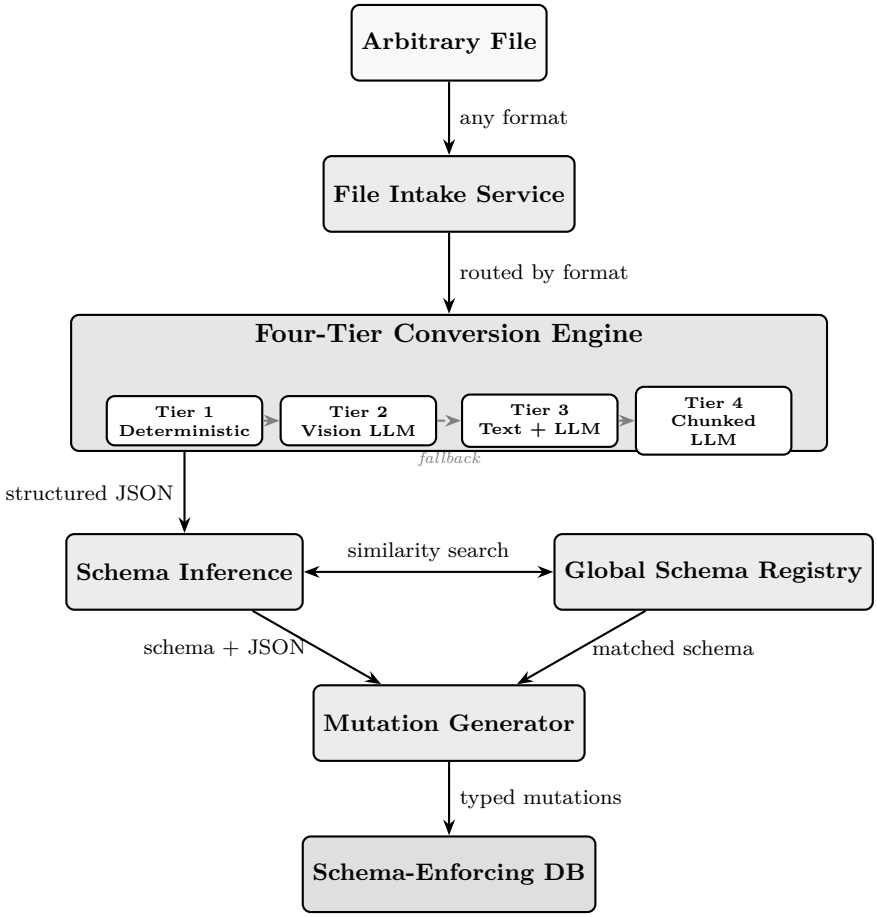

Figure 1: **System Architecture.** Six components: file intake service, four-tier conversion engine, schema inference module, global schema registry, mutation generator, and schema-enforcing database. Files enter at the top, are converted to JSON by the appropriate tier, matched against existing schemas, and produce typed mutations for the database.

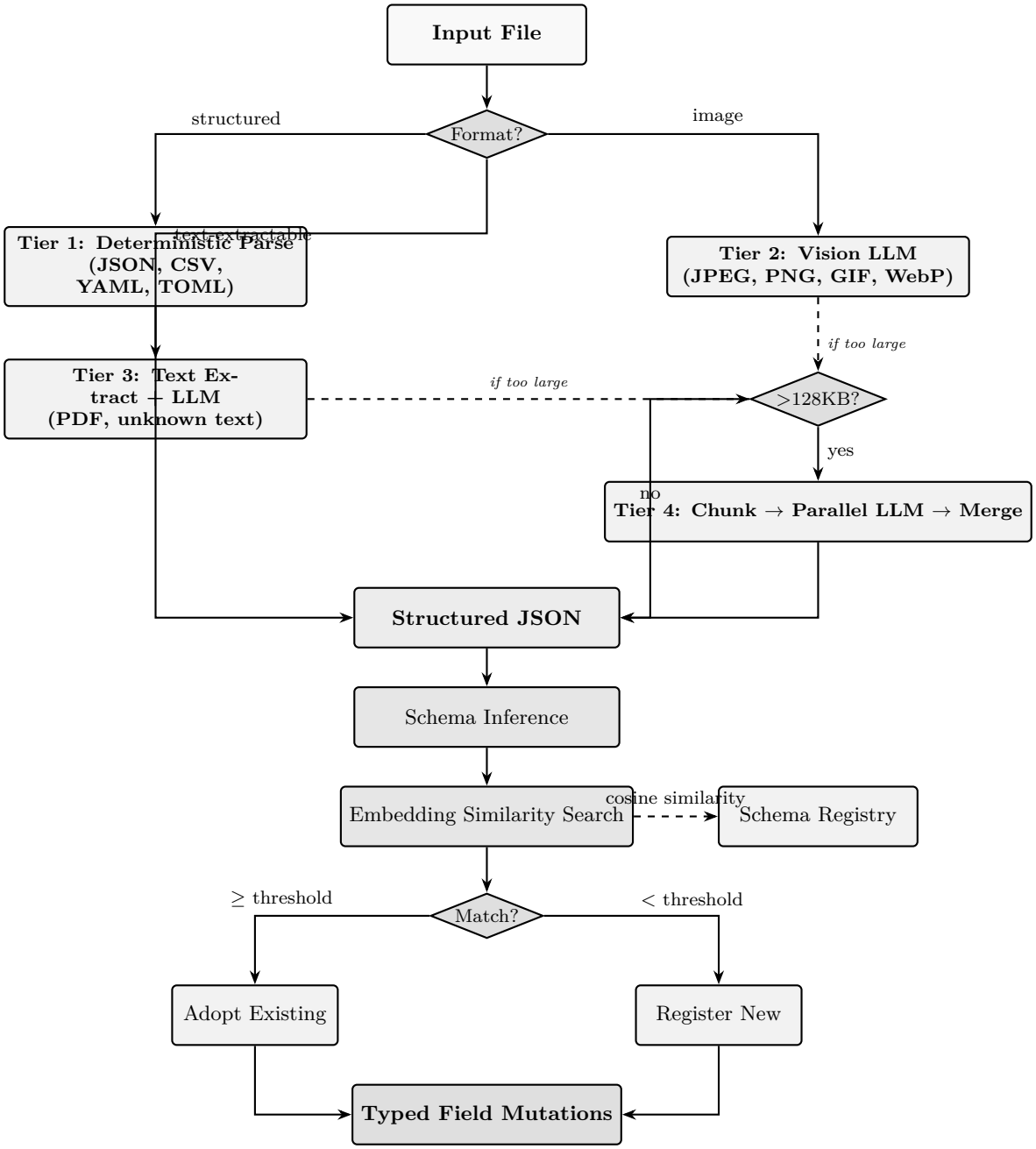

Figure 2: **Conversion Pipeline and Schema Matching Flow.** Files are routed to the appropriate conversion tier based on format. Large files fall through to Tier 4 for chunked parallel processing. The resulting JSON is analyzed for schema inference, compared against the global registry via embedding similarity, and produces typed mutations for the target schema.

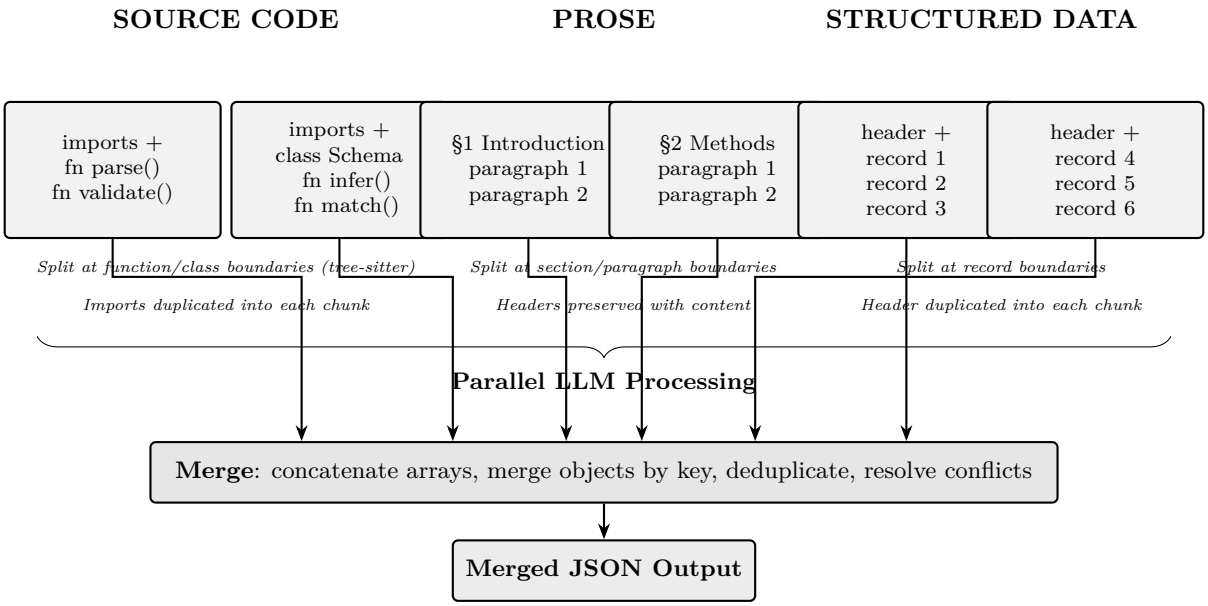

Figure 3: **Format-Aware Chunking Strategies.** Large files are split at semantically meaningful boundaries: function/class boundaries for source code (using tree-sitter), section/paragraph boundaries for prose, and record boundaries for structured data. Context (imports, headers) is duplicated into each chunk. Chunks are processed in parallel and merged with deduplication and conflict resolution.